

GridSense AI

Intelligent Electricity Demand & Peak Load Forecasting System for Delhi Power Grid

Enterprise Product Development Roadmap

Phased implementation plan for hourly demand forecasting, peak prediction, explainable AI, load reduction, power

Current prerequisite: the repository is empty and pnpm install fails because no package.json exists. Restore the intended re

Roadmap at a Glance

- 0 Project recovery - runnable frontend and backend workspace
- 1 Architecture and UX - approved routes, roles, and design system
- 2 Authentication - secure login and role-based access
- 3 Backend and database - versioned API and PostgreSQL foundation
- 4 Dataset management - validated, versioned load and weather data
- 5 Baseline forecasting - first reliable hourly forecast product
- 6 Advanced AI models - Transformer, LSTM, and XGBoost ensemble
- 7 Dashboard and forecast UX - operational monitoring and forecast workflows
- 8 Peak prediction and alerts - peak-risk monitoring and notifications
- 9 Explainable AI - SHAP-based feature and prediction explanations
- 10 Recommendation engine - commercial load and power purchase actions
- 11 Reports and research - exports, analytics, and experiment comparison
- 12 Administration and security - enterprise controls and auditing
- 13 Testing - automated confidence across the system
- 14 Deployment - production-ready containers and operations

Phase 0: Project Recovery and Workspace Setup

Restore or create the application foundation and make the development environment runnable.

- Add package.json and pnpm workspace configuration.
- Set up Next.js, TypeScript, Tailwind CSS, and reusable UI foundations.
- Set up the FastAPI backend and PostgreSQL configuration.
- Add an environment variable template and Docker Compose setup.
- Configure Builder setup command: pnpm install.
- Configure Builder dev command: pnpm dev.
- Recommended structure: frontend, backend, ai, database, docker, tests,
- docker-compose.yml, README.md.

Phase 1: Product Architecture and UX Design

Define the page map, user journeys, permissions, responsive layout, and visual system.

- Pages: Landing, Login, Dashboard, Forecast, Peak Prediction, AI Explanation, Recommendations, Alerts, Analytics, Datasets, Model Training, Model Comparison, Research, Reports, Profile, Settings, and Admin Panel.
- Create a page and routing map and a role-permission matrix.
- Build the responsive application shell and sidebar.
- Create design tokens and reusable cards, tables, charts, forms, modals, and alerts.
- Define loading, empty, and error states.

Phase 2: Landing Website and Authentication

Build the public landing experience and secure user access.

- Landing sections: Hero, Features, AI Engine, Technology, Architecture, Research, Contact, and Request Demo.
- Implement login, logout, password hashing, JWT access tokens, and refresh-token handling.
- Protect frontend routes and enforce authorization in the backend.
- Support Super Admin, Grid Operator, and Research Analyst roles.
- Acceptance: users authenticate safely, protected pages reject unauthenticated access, and each role sees only permitted modules.

Phase 3: Database and Backend Foundation

Create the FastAPI service, PostgreSQL schema, migrations, and API contracts.

- Core tables: Users, Roles, HistoricalLoad, Weather, Calendar, Predictions,
- Recommendations, Reports, Alerts, ResearchResults, and TrainingLogs.
- Use SQLAlchemy models, Pydantic schemas, service and repository layers, versioned
- APIs, structured errors, and audit logging.
- API groups: auth, users, dashboard, forecast, peaks, explanations,
- recommendations, datasets, models, reports, research, and alerts.

Phase 4: Dataset Management

Build reliable ingestion for historical load, weather, calendar, holidays, festivals, and related predictors.

- Support CSV upload, preview, column mapping, and dataset versioning.
- Validate date and time, missing values, duplicates, units, and schema.
- Generate a data-quality report and preserve import history.
- Store historical load, temperature, rainfall, weekend, holiday, festival, previous-day, and previous-week features.
- Acceptance: invalid datasets are rejected with clear messages; valid datasets can be stored, queried, and reviewed before training.

Phase 5: Baseline Forecasting Pipeline

Deliver a dependable forecasting product before adding complex deep-learning models.

- Pipeline: historical data -> cleaning -> feature engineering -> train/test split
- -> baseline model -> forecast -> evaluation.
- Initial models: persistence, moving average, linear regression, Random Forest, and
- XGBoost.
- Outputs: hourly demand, peak demand, peak timing, confidence score, forecast
- horizon, and selected model.
- Metrics: MAE, RMSE, MAPE, R2, peak timing error, and peak magnitude error.

Phase 6: Advanced AI Models

Implement the hybrid architecture incrementally and keep each training run reproducible.

- Target pipeline: historical load -> feature engineering -> Transformer -> LSTM ->
- XGBoost -> forecast ensemble -> SHAP -> recommendation engine.
- Use Transformer models for temporal patterns, LSTM for sequential dependencies,
- and XGBoost for feature-based correction.
- Add an ensemble or stacking layer, model versioning, and artifact storage.
- Support training date range, horizon, feature, and hyperparameter controls.
- Store training progress, validation metrics, and deployment status.

Phase 7: Dashboard and Forecast Experience

Connect operational screens to backend APIs.

- Dashboard cards: Current Load, Today's Peak, Tomorrow's Peak, Temperature,
- Rainfall, Prediction Accuracy, Confidence, Commercial Load, Active Alerts, and
- Recommendations.
- Charts: hourly, weekly, monthly, and yearly demand; peak history; actual versus
- predicted load; confidence intervals.
- Forecast workflow: date, range, and model filters; hourly forecast table; peak
- details; and export action.
- Acceptance: charts use API data, filters update results, and empty or failed
- responses are visible and understandable.

Phase 8: Peak Prediction and Alerts

Monitor peak-load risk and notify operators before critical periods.

- Show today's and tomorrow's peak, peak duration, timing, probability, and severity.
- Create high-demand, low-confidence, forecast-deviation, data-quality, and model-performance alerts.
- Support severity, acknowledgement, filtering, history, and role-based visibility.

Phase 9: Explainable AI

Make each prediction understandable to grid operators and researchers.

- Provide global and per-prediction SHAP feature importance.
- Show positive and negative feature effects.
- Generate a natural-language explanation.
- Support model explanation comparison.
- Explain the impact of temperature, previous load, calendar, rainfall, festivals, and hour of day.
- Acceptance: every forecast links to an explanation generated from the selected model and prediction.

Phase 10: Recommendation Engine

Convert forecasts into operational actions.

- Recommendation categories: commercial load reduction, power purchase planning, demand response, and peak management.
- Recommendation fields: priority, action, expected load reduction, expected savings, time window, confidence, reason, and status.
- Identify high-demand windows, estimate controllable commercial load, recommend reduction periods, estimate savings, recommend additional power purchase, and track completion.

Phase 11: Reports, Analytics, and Research

Add reporting, exports, analytics, and model-analysis capabilities.

- Reports: daily demand, weekly forecast, monthly peak, model performance, savings, and data quality.
- Exports: PDF, CSV, Excel-compatible data, and JSON.
- Research dashboard: dataset versions, experiment history, model comparison, metrics, visualizations, reproducibility metadata, and research-result exports.
- Compare accuracy, training duration, inference time, peak accuracy, confidence, and SHAP results.

Phase 12: Administration, Security, and Auditing

Complete enterprise administration and security controls.

- Provide user management, role assignment, account activation, and deactivation.
- Control dataset permissions, model deployment, and system settings.
- Store audit logs, training logs, and API activity logs.
- Use JWT expiration, backend authorization, input and upload validation, rate limiting, restricted CORS, secure secrets, safe logging, backups, and recovery.

Phase 13: Testing and Quality Assurance

Implement testing throughout the project rather than waiting until the end.

- Frontend: components, validation, route protection, charts, and role navigation.
- Backend: authentication, RBAC, APIs, repositories, dataset validation, forecasting, and recommendations.
- AI: feature engineering, leakage prevention, output shapes, metrics, reproducibility, and SHAP.
- Integration: login to dashboard, upload to forecast, training to comparison, forecast to recommendation, and report generation.

Phase 14: Deployment and Operations

Prepare production deployment using containers and operational safeguards.

- Create frontend and backend Dockerfiles.
- Configure PostgreSQL, NGINX reverse proxy, and production environment settings.
- Add health checks, migrations, logging, monitoring, backups, and recovery.
- Automate the delivery sequence: build -> run tests -> build Docker images -> apply migrations -> deploy backend -> deploy frontend -> run health checks.

Recommended MVP Sequence

1. Project setup
2. Authentication and RBAC
3. Database foundation
4. Dataset upload
5. Baseline forecasting
6. Dashboard
7. Peak prediction
8. Basic recommendations
9. Reports
10. Testing and deployment

After the MVP is stable, add the Transformer, LSTM, hybrid ensemble, SHAP explanations, advanced research t